SLSA Implementation Checkpoint Guide

A structured 5-step guide to implement your self-learning assistant with visible wins at each checkpoint. Each step takes less than 20 minutes and provides immediate feedback.

🎯 Overview

This guide walks you through building a complete self-learning assistant system from scratch. Each checkpoint builds on the previous one and delivers a working component you can see and test.

Total Time: ~75 minutes (15min per checkpoint)
Skill Level: Beginner to Intermediate
Prerequisites: Python 3.8+, basic command line knowledge

📋 Checkpoint 1: Setup & Validation (5 minutes)

Goal: Verify all components are installed and working
Visible Win: See the MCP server respond to health checks

Steps:

1. **Install Dependencies**
   ```
   bash
   pip install fastapi uvicorn requests tkinter
   ```

2. **Start the MCP Server**
   ```
   bash
   python assistant/mcp_simple_server.py
   ```

3. **Verify Server is Running**
   ```
   bash
   curl http://localhost:8000/status
   ```

Expected Output:
```
json
{
  "status": "healthy",
  "timestamp": "2024-01-01T12:00:00",
  "learning_sessions": 0,
  "total_insights": 0
}
```

1. **Check API Documentation**
   - Open browser: http://localhost:8000/docs
   - You should see interactive API documentation

✅ Success Criteria:

- [ ] Server starts without errors

- [ ] `/status` endpoint returns healthy response
- [ ] API docs are accessible
- [ ] No error messages in terminal

🔧 Troubleshooting:

- **Port 8000 in use**: Change port in `mcp_simple_server.py` line 156
- **Module not found**: Run `pip install -r requirements.txt`
- **Permission denied**: Use `python3` instead of `python`

📋 Checkpoint 2: First Learning Cycle (10 minutes)

Goal: Process your first document and see insights generated
Visible Win: Watch the system extract 3 insights from content

Steps:

1. **Create a Test Document**
   `bash`
   ```
   echo "Artificial Intelligence is transforming how we process data and learn from information. Machine learning models can analyze patterns and generate insights automatically. This technology enables systems to improve their performance over time through continuous learning cycles." > test_document.txt
   ```

2. **Process the Document**
   `bash`
   ```
   python assistant/knowledge_simple.py test_document.txt
   ```

Expected Output:
```
🧠 Knowledge Ingestion System
   ========================================
   📄 Processing: test_document.txt
   ✅ Success! Generated 3 insights:
      1. This is a concise piece of content suitable for quick processing.
      2. Content focuses on technical concepts: AI, data, learn
      3. Content is presented as continuous text.
   🎯 Confidence: 0.85
```

1. **Verify Learning Data**
   `bash`
   ```
   curl http://localhost:8000/insights
   ```

2. **Check Updated Metrics**
   `bash`
   ```
   curl http://localhost:8000/metrics
   ```

✅ Success Criteria:

- [ ] Document processing completes successfully
- [ ] 3 insights are generated and displayed
- [ ] Server metrics show 1 learning session
- [ ] Insights are retrievable via API

🔧 Troubleshooting:

- **Server not responding**: Restart MCP server from Checkpoint 1
- **No insights generated**: Check server logs for errors
- **File not found**: Ensure test_document.txt was created correctly

---

📋 Checkpoint 3: Dashboard Integration (15 minutes)

Goal: Launch the visual dashboard and see real-time metrics
Visible Win: Interactive dashboard showing learning progress

Steps:

1. **Launch the Dashboard** (in a new terminal)
   `bash`
   `python tk_simple_dashboard.py`

2. **Verify Dashboard Components**
   - Server Status: Should show "🟢 Server Online"
   - Learning Sessions: Should show "1"
   - Total Insights: Should show "3"
   - Recent Insights: Should display your generated insights

3. **Test the Learning Button**
   - Click "🧪 Test Learning" button
   - Confirm success dialog appears
   - Watch metrics update in real-time

4. **Process Another Document**
   `bash`
   `echo "Data science involves collecting, cleaning, and analyzing large datasets to extract meaningful patterns. Statistical methods and visualization techniques help researchers understand complex relationships in data." > data_science.txt`
   `python assistant/knowledge_simple.py data_science.txt`

5. **Watch Dashboard Update**
   - Click "🔄 Refresh Now" or wait for auto-update
   - Verify metrics increase
   - See new insights appear

✅ Success Criteria:

- [ ] Dashboard opens without errors
- [ ] All metrics display correctly
- [ ] Test learning button works
- [ ] Dashboard updates with new data
- [ ] Recent insights section shows content

🔧 Troubleshooting:

- **Dashboard won't open**: Install tkinter: `sudo apt-get install python3-tk`
- **Connection failed**: Verify MCP server is running on port 8000

- **Metrics not updating**: Check server URL in dashboard (bottom right)

📋 Checkpoint 4: Automation Setup (20 minutes)

Goal: Set up automated workflow with n8n
Visible Win: Automated document processing pipeline

Steps:

1. **Install n8n** (if not already installed)
   ```
   bash
      npm install -g n8n
      # OR using Docker:
      docker run -it --rm --name n8n -p 5678:5678 n8nio/n8n
   ```

2. **Start n8n**
   ```
   bash
      n8n start
   ```
   - Open browser: http://localhost:5678

3. **Import the Workflow**
   - Click “Import from file”
   - Select `n8n_workflow.json`
   - The workflow should load with 3 nodes:
     - File Trigger
     - HTTP Request (to MCP server)
     - Data Processing

4. **Configure the Workflow**
   - Click on “HTTP Request” node
   - Verify URL is set to: `http://localhost:8000/learn`
   - Set method to POST

5. **Test the Workflow**
   - Click “Execute Workflow”
   - Upload a test file or use manual trigger
   - Verify data flows through all nodes

6. **Set up File Monitoring**
   - Create a `watch_folder` directory
   - Configure File Trigger to monitor this folder
   - Test by dropping a file in the folder

✅ Success Criteria:

- [ ] n8n interface is accessible
- [ ] Workflow imports successfully
- [ ] Manual execution works
- [ ] File monitoring triggers workflow
- [ ] Data reaches MCP server

🔧 Troubleshooting:

- **n8n won't start**: Check if port 5678 is available
- **Workflow import fails**: Verify JSON file is valid
- **HTTP requests fail**: Ensure MCP server is running and accessible

---

📋 Checkpoint 5: Production Deploy (15 minutes)

Goal: Deploy the complete system using Docker
Visible Win: Full system running in production mode

Steps:

1. **Configure Environment**
   ```
   bash
   cp .env.example .env
   # Edit .env with your settings (optional for basic setup)
   ```

2. **Build and Start Services**
   ```
   bash
   docker-compose up -d
   ```

3. **Verify All Services**
   ```
   bash
   docker-compose ps
   ```

Expected Output:
```
NAME                    STATUS
  slsa-mcp-server        Up
  slsa-dashboard         Up (if included)
```

1. **Test Production Endpoints**
   ```
   bash
   curl http://localhost:8000/status
   curl http://localhost:8000/metrics
   ```

2. **Process a Document in Production**
   ```
   bash
   echo "Production test document with advanced AI concepts and machine learning algorithms for automated data processing." > prod_test.txt
   python assistant/knowledge_simple.py prod_test.txt
   ```

3. **Monitor Logs**
   ```
   bash
   docker-compose logs -f mcp-server
   ```

4. **Test Scaling** (Optional)
   ```
   bash
   docker-compose up -d --scale mcp-server=2
   ```

✅ Success Criteria:

- [ ] Docker services start successfully

- [ ] All endpoints respond correctly
- [ ] Document processing works in production
- [ ] Logs show healthy operation
- [ ] System handles multiple requests

🔧 Troubleshooting:

- **Docker build fails**: Check Docker is installed and running
- **Port conflicts**: Modify ports in docker-compose.yml
- **Service won't start**: Check logs with `docker-compose logs [service]`

🎉 Completion Checklist

After completing all checkpoints, you should have:

- [ ] **Working MCP Server**: Responds to all API endpoints
- [ ] **Knowledge Processing**: Can extract insights from documents
- [ ] **Visual Dashboard**: Real-time metrics and monitoring
- [ ] **Automation Pipeline**: n8n workflow for automated processing
- [ ] **Production Deployment**: Docker-based scalable system

🚀 Next Steps

1. **Customize for Your Domain**: Modify insight generation logic
2. **Add More Data Sources**: Extend beyond PDF processing
3. **Enhance UI**: Build web-based dashboard
4. **Scale Up**: Add database, caching, load balancing
5. **Monitor**: Set up logging, alerting, and analytics

📞 Support

If you encounter issues:
1. Check the troubleshooting sections above
2. Review logs for error messages
3. Verify all prerequisites are installed
4. Test each component individually

Remember: Each checkpoint should give you a visible, working result. Don't move to the next checkpoint until the current one is fully working!